

3.6 - APIs REST para escala

O contrato que cresce sem virar dívida

Três dias só para entender a API

- **API de integração cresceu organicamente por dezoito meses**
 - Endpoints nomeados por ação, não por recurso

LUNALOG

A API que a LunaLog oferece às transportadoras cresceu sem plano por dezoito meses. Tinha endpoints como POST /fazer-cotacao, GET /pegar-status-da-entrega e POST /confirmar-tudo, cada um com sua própria lógica de nomes. Um desenvolvedor de uma nova transportadora levava três dias só para entender como aquilo funcionava, antes de escrever uma linha de integração. O redesign começou pelo vocabulário, e foi aí que ficou óbvio: aquela bagunça não era inevitável, era o resultado de não ter pensado o contrato desde a primeira versão.

Pilares de um REST maduro

Modelar recursos, não operações

- **Verbos HTTP carregam a intenção, não a URL**
 - GET lê, POST cria, PUT substitui, DELETE remove
- **Recursos são substantivos, nunca ações no caminho**
 - Use /cotacoes e /entregas, não /fazer-cotacao
- **Status code semântico comunica o resultado real**
 - 2xx sucesso, 4xx erro do cliente, 5xx erro do servidor
- **Paginação e versionamento desde a primeira versão**
 - Cursor ou offset para grandes conjuntos, versão no path ou header

Antes e depois do redesign

Contrato problemático

- Ação no nome: POST /fazer-cotacao
- Erro retornado com status 200
- Resposta inteira sem paginação
- Sem versionamento, quebra a cada mudança
- Cada endpoint com convenção própria

Contrato bem modelado

- Recurso no nome: POST /cotacoes
- Erro com status 4xx ou 5xx correto
- Listagem paginada por padrão
- Versão explícita protege integrações
- Convenção única e previsível em tudo

Idempotência é propriedade de design

- **Operação idempotente repetida produz o mesmo resultado**
- **Cliente que faz retry não deve criar duplicatas**

Aceite uma chave de idempotência no header de operações de criação. Assim, um retry após timeout não vira uma segunda cobrança ou uma confirmação duplicada.



REST em profundidade resolve muito bem o contrato público. Mas entre os seus próprios serviços, no fundo da arquitetura, REST nem sempre é a melhor escolha. Usar REST para tudo é como usar uma chave de fenda para todo tipo de parafuso. Existem duas alternativas que brilham em contextos opostos.

gRPC e GraphQL: a ferramenta certa por camada